

Probabilistic Graphical Models

CSE 5830

Inference

Lecture 9

**Exact inference: Variable
Elimination and clique graph**

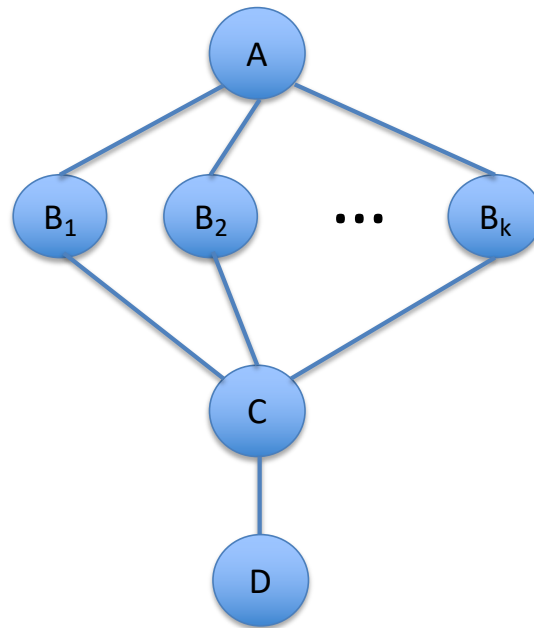
Goals for Today

- Elimination as graph operation
 - Induced graph
- Exact inference using Clique tree
 - Cluster graph
 - Properties of cluster graph

Variable elimination

What is an appropriate order for variable elimination to obtain $P(D)$?

- a) $A, B_1, \dots, B_K, C$
- b) $A, C, B_1, \dots, B_K$
- c) $C, B_1, \dots, B_K, A$
- d) $B_1, \dots, B_K, A, C$



PGM Quiz 3

<https://forms.cloud.microsoft/r/VM7Lx9MSFA>



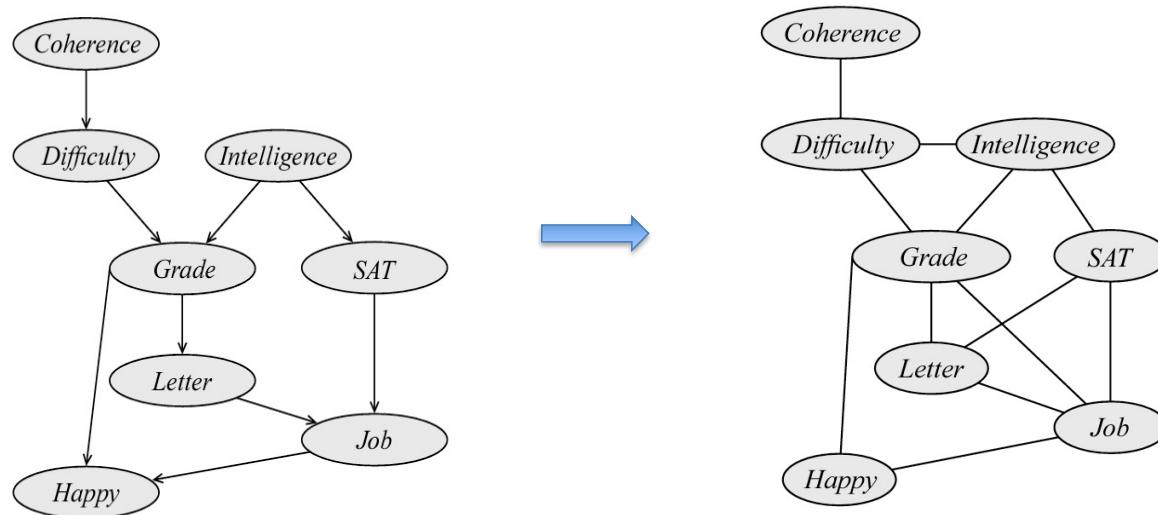
Graph-based perspective

- Can we implement VE on graph?
- VE algorithm operates on undirected graph H_Φ , $\Phi = \{\phi\}$
- Nodes of $H_\Phi \rightarrow$ variables in $\text{scope}[\Phi]$
- Edge $X_i - X_j$ in $H_\Phi \iff$ there is ϕ in Φ , such that X_i, X_j are in $\text{scope}[\phi]$
 - Fully connected over the $\text{scope}[\phi]$
- For BNs, H_Φ should be a moralized graph
- Each VE step is a transformation on the graph.

Elimination as graph operation

BN $\rightarrow$ MN $\rightarrow$ Moralized graph

$$\phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_G(G, I, D) \phi_H(H, G, J) \phi_I(I) \phi_D(C, D) \phi_C(C)$$

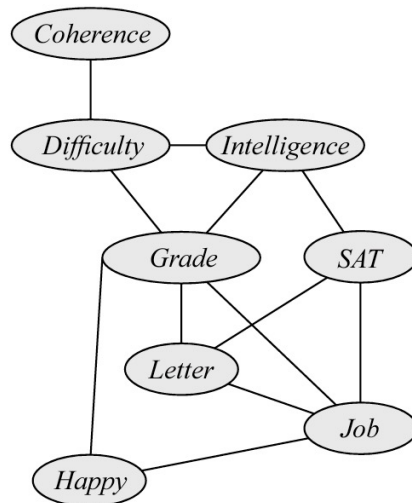


Elimination as graph operation

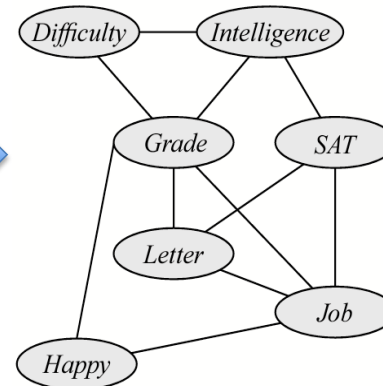
Eliminating C

$$\phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_G(G, I, D) \phi_H(H, G, J) \phi_I(I) \phi_D(C, D) \phi_C(C)$$

New factor $\rightarrow \tau_1(D) = \sum_C \underbrace{\phi_D(C, D) \phi_C(C)}_{\text{intermediate factor}}$



Induced MN after eliminating C

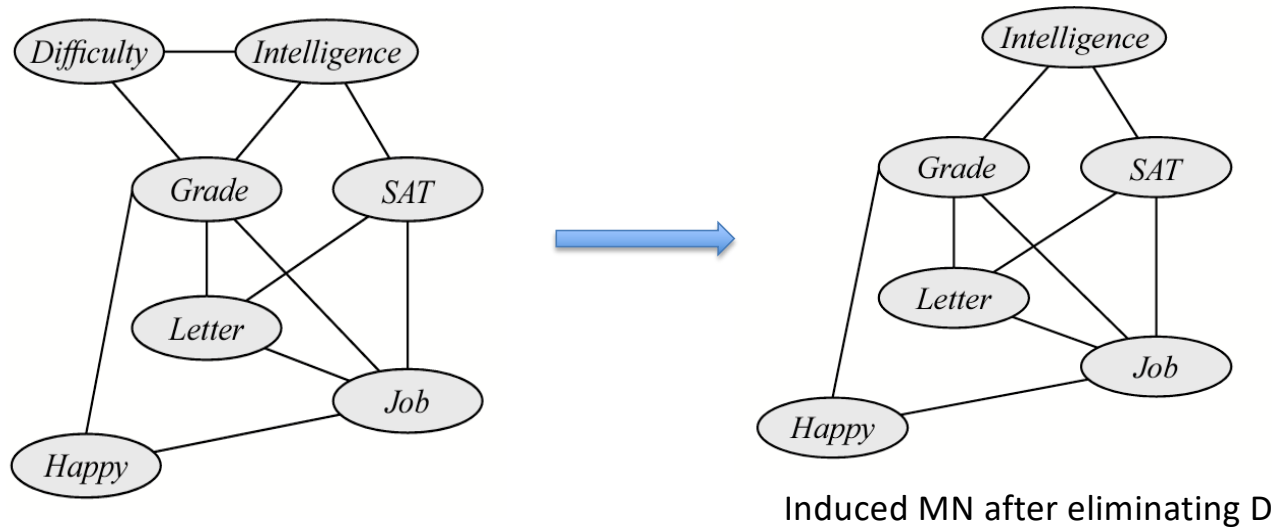


Elimination as graph operation

Eliminating D

$$\tau_2(G, I) = \sum_D \phi_G(G, I, D) \tau_1(D)$$

$$\phi_J(J, L, S) \phi_L(L, G) \phi_S(S, I) \phi_H(H, G, J) \phi_I(I) \tau_2(G, I)$$



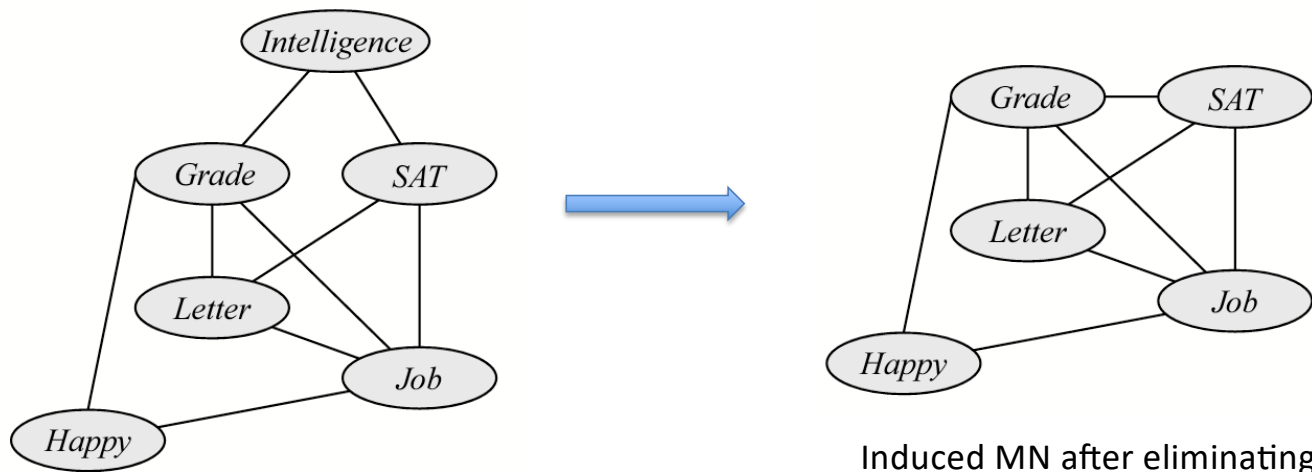
Induced MN after eliminating D

Elimination as graph operation

Eliminating I

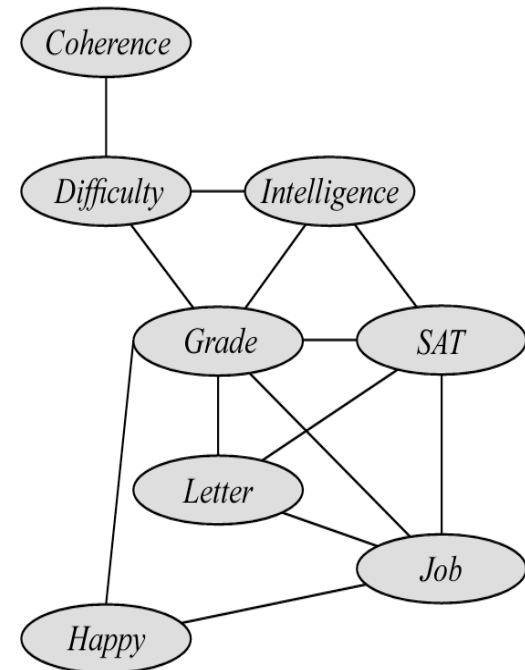
$$\tau_3(S, G) = \sum_I \phi_S(S, I) \phi_I(I) \tau_2(G, I)$$

$$\phi_J(J, L, S) \phi_L(L, G) \phi_H(H, G, J) \tau_3(S, G)$$



Induced Graph

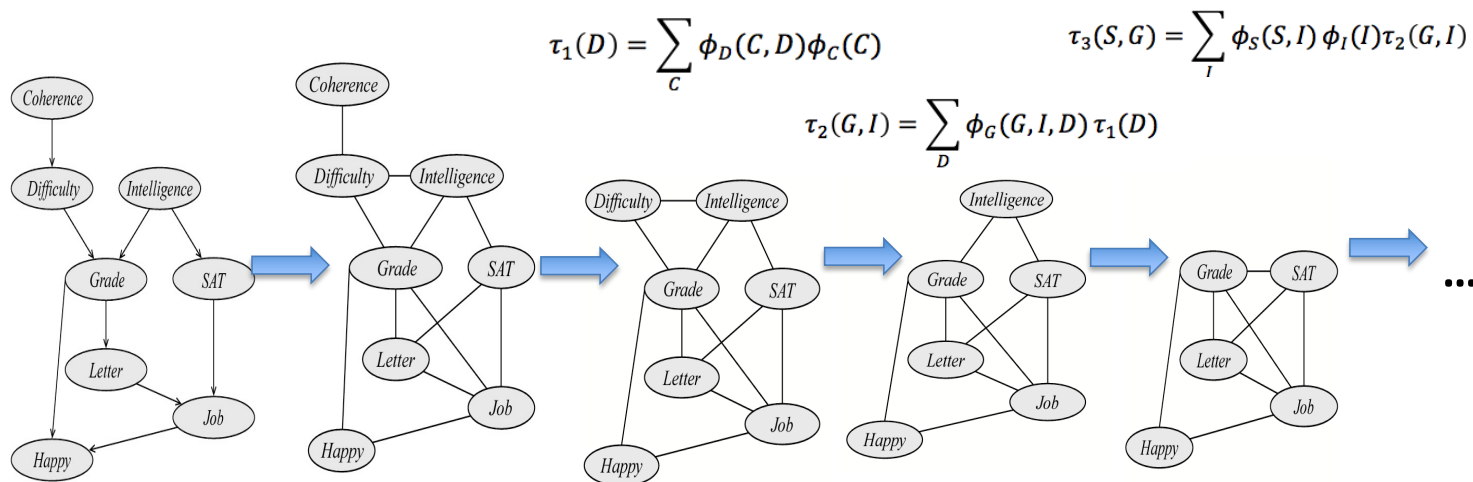
- Take the union over all of the undirected graphs from each step
→ induced graph
 - The scope of every intermediate factor is a clique in this graph.



different ordering → different induced graph

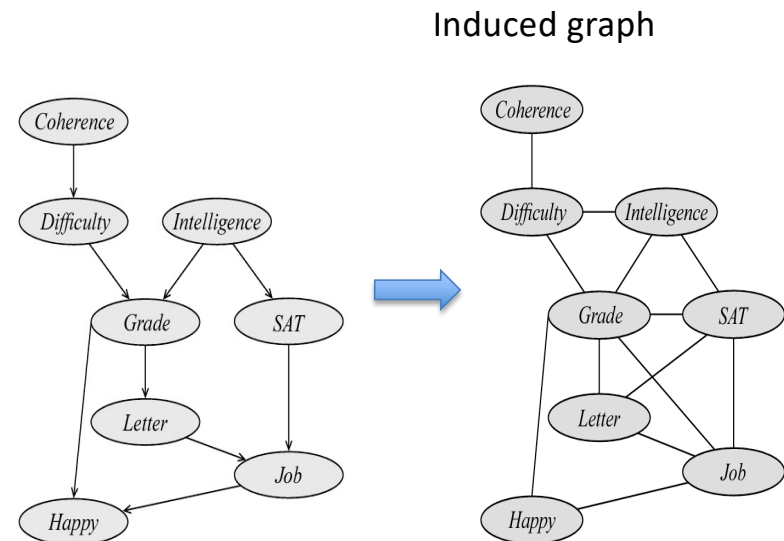
Induced graph

Combining all the graphs



Induced Graph

- The induced graph $I_{\Phi, \alpha}$ over factors Φ and ordering α :
 - Undirected graph
 - X_i and X_j are connected if they appeared in the same factor in a run of the VE algorithm using α as the ordering



How to identify intermediate factors in the VE algorithm from the graph?

Cliques in the Induced Graph

Every maximal clique in the induced graph is the scope of some intermediate factors.

$$\tau_1(D) = \sum_C \phi_C(C) \phi_D(C, D)$$

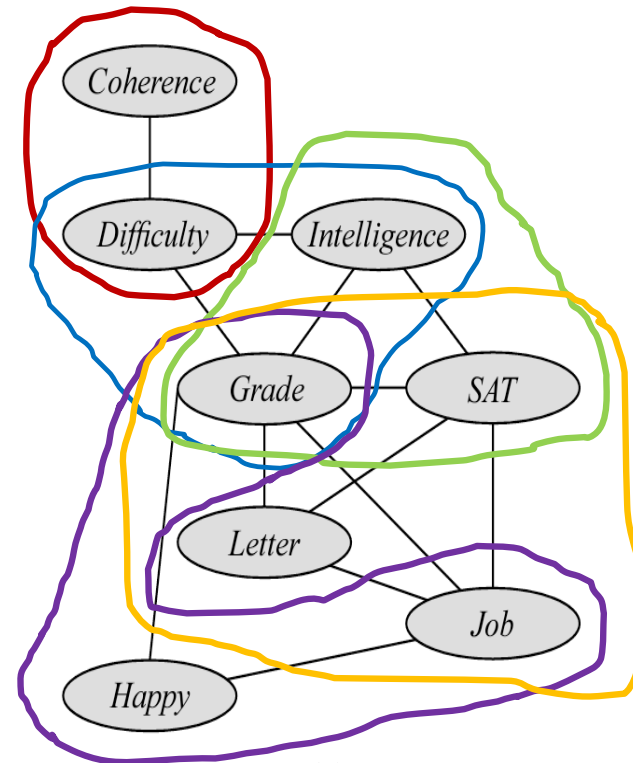
$$\tau_2(G, I) = \sum_D \phi_G(G, I, D) \tau_1(D)$$

$$\tau_3(S, G) = \sum_I \phi_S(S, I) \phi_I(I) \tau_2(G, I)$$

$$\tau_4(G, J) = \sum_H \phi_H(H, G, J)$$

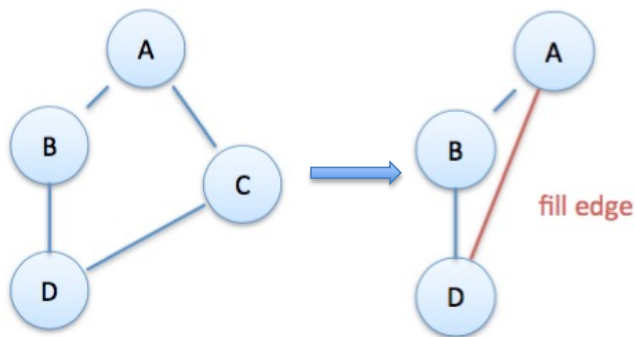
$$\tau_5(L, J) = \sum_G \phi_L(L, G) \tau_3(S, G) \tau_4(G, J)$$

$$\tau_6 = \sum_{L, S} \phi_J(J, L, S) \tau_5(L, J)$$



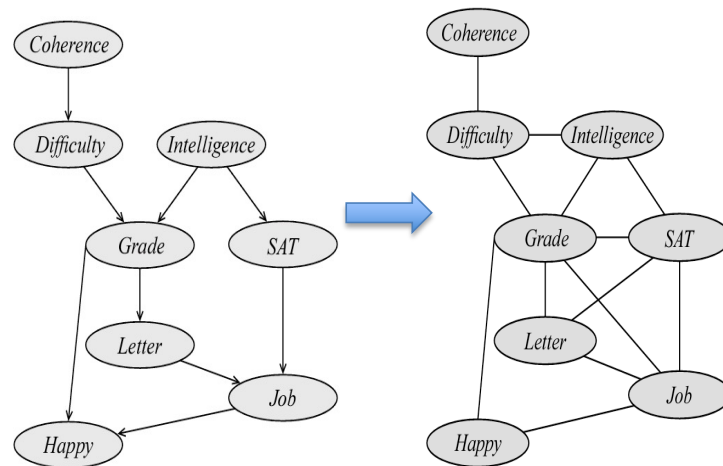
Induced graph and chordal Graphs

- Chordal graph: Undirected graph whose minimal cycles are not longer than 3.
- Induced graphs are always chordal



Eliminating C

Generating a new factor $\rightarrow$ adding fill edge



Elimination as graph operation

- Variable elimination can be viewed as transformations on undirected graph
 - Elimination connects all node's current neighbors
- Cliques in resulting induced graph directly correspond to algorithm's complexity

How can we estimate complexity using graphs?

Definitions: Induced Width and Tree Width

- Induced Width: Number of nodes in the largest clique of the induced graph, minus one.
 - Relative to an ordering
- Tree width: minimum width over all possible orderings.
 - Bound on the best performance we can hope for
- VE runtime is exponential in tree width

How can we find the best ordering?

Finding Elimination Orderings

Theorem: For a graph H , determining whether there exists an elimination ordering for H with induced width $\leq K$ is NP-complete

- This NP-hardness result is distinct from the NP-hardness result of inference
 - Even given the optimal ordering, inference may still be exponential

Finding Elimination Orderings

- Greedy search using heuristic cost function
 - At each point, eliminate node with smallest cost
- Possible cost functions:
 - min-neighbors: # neighbors in current graph
 - min-weight: weight (# values) of factor formed
 - min-fill: number of new fill edges
 - weighted min-fill: total weight of new fill edges (edge weight = product of weights of the 2 nodes)

Finding Elimination Orderings

- Can find elimination ordering by finding a low-width triangulation of original graph H_Φ
- Chordal graphs always admit an elimination ordering that doesn't introduce any fill edges into the graph.
 - No fill edges: no blowup.
 - Inference becomes linear in size of the factors already present!

VE shortcoming

- Finding the optimal elimination ordering is NP-hard
 - Simple heuristics that try to keep induced graph small often provide reasonable performance
- If we want to ask the model for another query, we need to restart the algorithm from scratch. This is very computationally expensive.
 - Saving intermediate factors

Alternative implementation for VE

- Set of factors $\Phi = \{\phi_i\}$
- Set of random variables $\mathbf{X} = \{X_i\}$
- Scope of ϕ_i is X_i

$$\tilde{P}_{\Phi}(\mathcal{X}) = \prod_{\phi_i \in \Phi} \phi_i(\mathbf{X}_i)$$

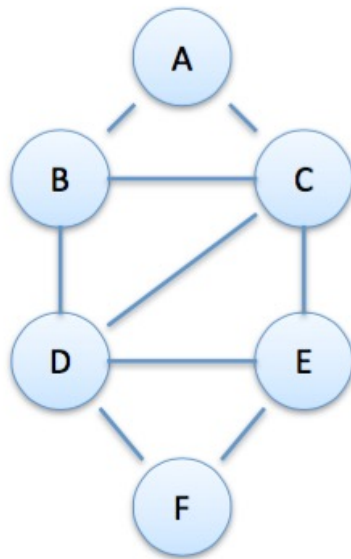
- VE as Passing message $\rightarrow$ Clique tree
 - ψ_i s take messages from other factors and generate new messages, τ_i s.
- Clique tree: Data structure provides a graphical flowchart of the factor manipulation process

Clique tree, definitions

- A clique is a subset of the nodes such that every two distinct nodes are adjacent.
 - A graph induced by cliques is a complete graph.
- A maximal clique is a clique that cannot be extended by including one more adjacent node
- Clique tree is a tree of cliques
 - Every maximal clique becomes a node
 - Connect nodes with overlapping variables
 - If graph was triangulated, always get a tree structure

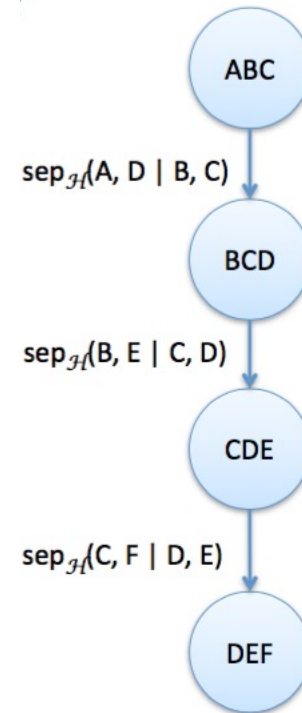
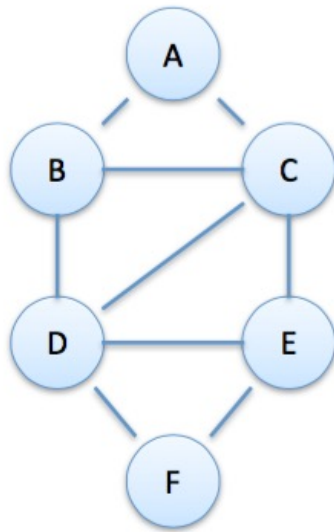
Clique tree example

- Every maximal clique becomes a node
- Connect nodes with overlapping variables



Clique tree

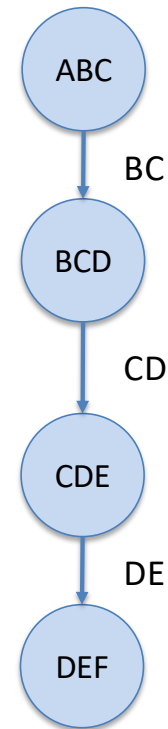
For each edge, intersection of random variables separates the rest in H .



Can be seen as a cluster graph

Cluster graph

- Graph of clusters connected by subsets of random variables in the scope of clusters, or sepsets
- Adjacent clusters pass information to each other about variables in sepset



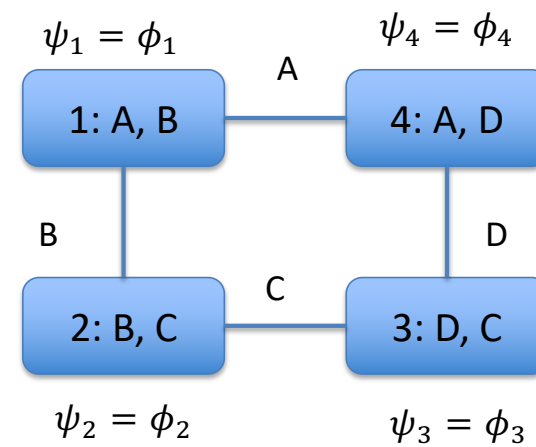
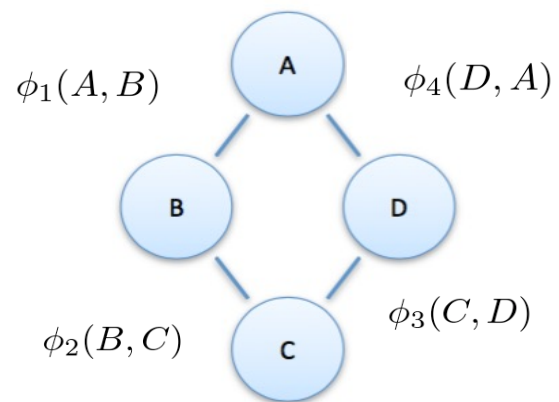
Cluster graph

- Undirected graph such that:
 - nodes are clusters $\mathcal{C}_i \subseteq \{X_1, \dots, X_n\}$
 - edge between $\mathcal{C}_i$ and $\mathcal{C}_j$ associated with sepset $\mathcal{S}_{i,j} \subseteq \mathcal{C}_i \cap \mathcal{C}_j$
- Given set of factors Φ , we assign each ϕ_k to a cluster $\mathcal{C}_{\alpha(k)}$ s.t. $\text{Scope}[\phi_k] \subseteq \mathcal{C}_{\alpha(k)}$

- Define

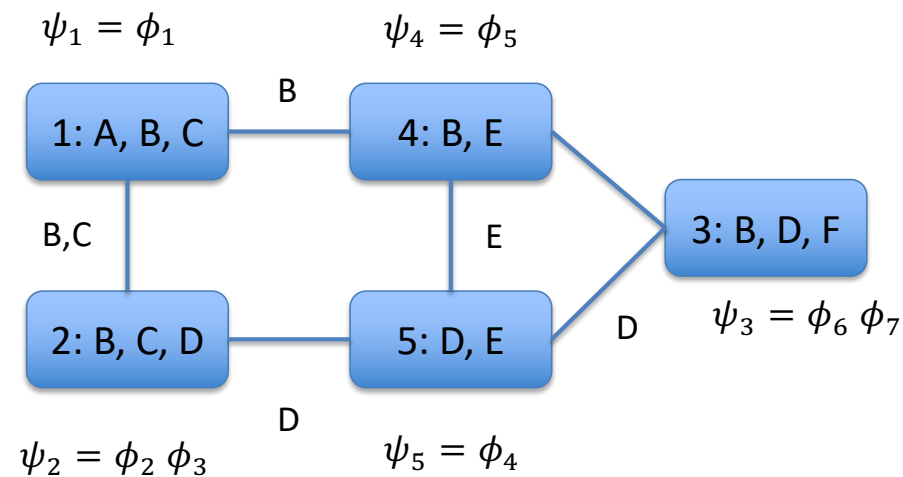
$$\psi_i(\mathcal{C}_i) = \prod_{k:\alpha(k)=i} \phi_k$$

Cluster graph example



Cluster graph example

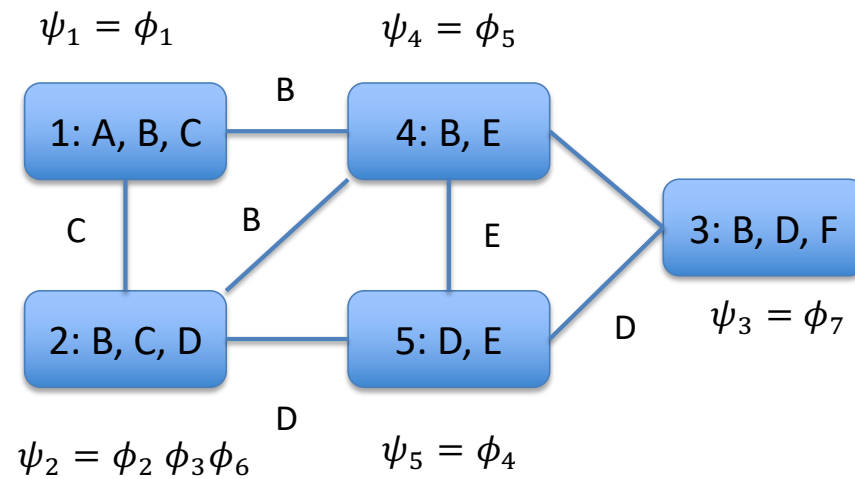
$\phi_1(A, B, C), \phi_2(B, C), \phi_3(B, D), \phi_4(D, E), \phi_5(B, E), \phi_6(B, D), \phi_7(B, D, F)$



Cluster graph example

- Build another cluster graph

$\phi_1(A, B, C), \phi_2(B, C), \phi_3(B, D), \phi_4(D, E), \phi_5(B, E), \phi_6(B, D), \phi_7(B, D, F)$



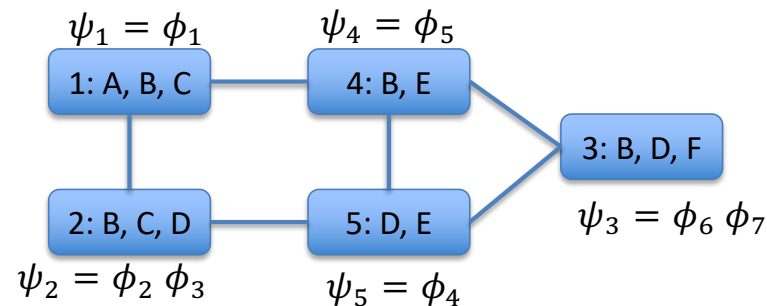
Cluster graph properties

- Family Preservation property
- Running Intersection Property

Family Preservation property

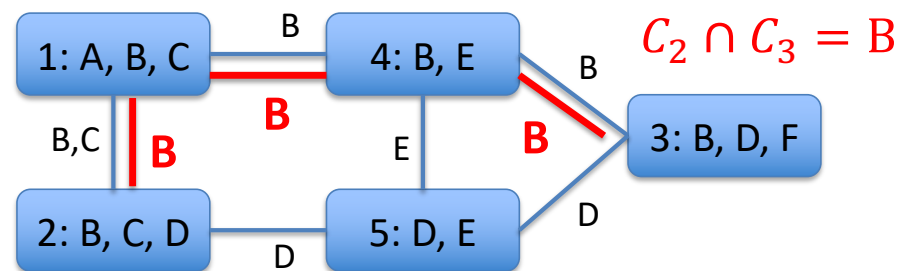
- Given set of factors Φ , we assign each ϕ_k to a cluster $\mathbf{C}_{\alpha(k)}$ s.t. $\text{Scope}[\phi_k] \subseteq \mathbf{C}_{\alpha(k)}$
- For each factor $\phi_k \in \Phi$, there exists a cluster $\mathbf{C}_i$ s.t. $\text{Scope}[\phi_k] \subseteq \mathbf{C}_i$

$\phi_1(A, B, C), \phi_2(B, C), \phi_3(B, D), \phi_4(D, E), \phi_5(B, E), \phi_6(B, D), \phi_7(B, D, F)$



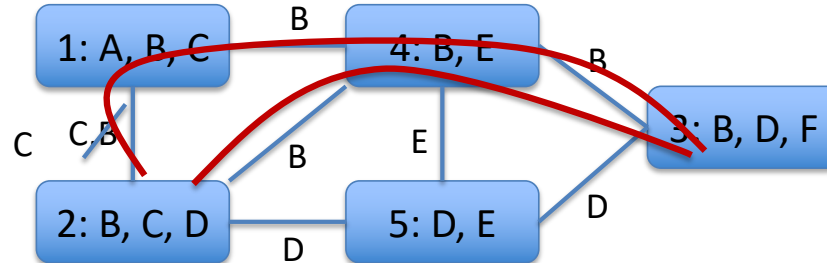
Running Intersection Property

- For each pair of clusters $\mathcal{C}_i, \mathcal{C}_j$ and variable $X \in \mathcal{C}_i \cap \mathcal{C}_j$ there exists a unique path between $\mathcal{C}_i$ and $\mathcal{C}_j$ for which all clusters and sepsets contain X



Running Intersection Property

- For any X , the set of clusters and sepsets containing X forms a tree



- “Cluster Tree” a cluster graph that is a tree.

VE in cluster graph

- Cluster graph with clusters $\mathcal{C}_i$
- ψ_i : intermediate factors for each cluster with $\mathcal{C}_i = \text{scope}[\psi_i]$

$$\psi_i(\mathcal{C}_i) = \prod_{k: \alpha(k)=i} \phi_k$$

- τ_i : new factor after eliminating a variable in ψ_i
- an edge between $\mathcal{C}_i$ and $\mathcal{C}_j$ if τ_i , called message, is used to compute ψ_i
 - Edges are annotated with direction

Example

$$\tau_1(D) = \sum_C \phi_C(C) \phi_D(C, D)$$

$$C_1 = \{C, D\}$$

$$\tau_2(G, I) = \sum_D \phi_G(G, I, D) \tau_1(D)$$

$$C_2 = \{G, I, D\}$$

$$\tau_3(S, G) = \sum_I \phi_S(S, I) \phi_I(I) \tau_2(G, I)$$

$$C_3 = \{S, I, G\}$$

$$\tau_4(G, J) = \sum_H \phi_H(H, G, J)$$

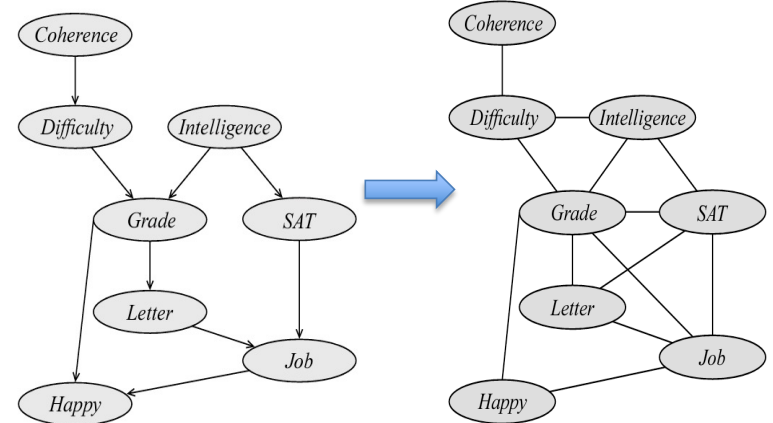
$$C_4 = \{H, G, I\}$$

$$\tau_5(S, L, J) = \sum_G \phi_L(L, G) \tau_3(S, G) \tau_4(G, J)$$

$$C_5 = \{L, G, S, J\}$$

$$\tau_6(L, J) = \sum_S \phi_J(J, L, S) \tau_5(S, L, J)$$

$$C_6 = \{J, L, S\}$$



Example

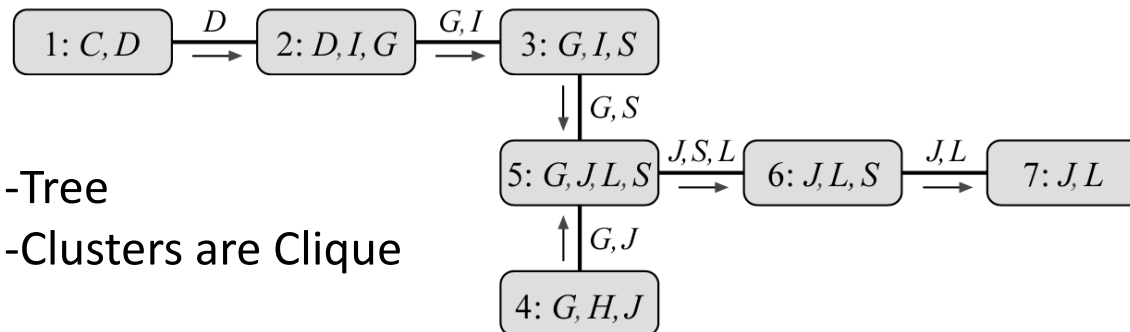
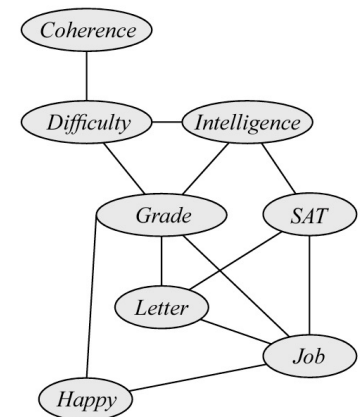
1-Start with undirected graph

2-Do VE

3-One node per C_i , edge $C_i - C_j$ when τ_i gets in computing ψ_j

- Execution of VE defines a direction for the edges

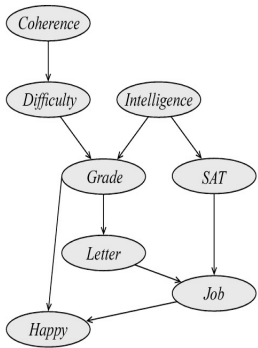
$$\begin{aligned}
 \tau_1(D) &= \sum_C \phi_C(C) \phi_D(C, D) & C_1 &= \{C, D\} \\
 \tau_2(G, I) &= \sum_D \phi_G(G, I, D) \tau_1(D) & C_2 &= \{G, I, D\} \\
 \tau_3(S, G) &= \sum_I \phi_S(S, I) \phi_I(I) \tau_2(G, I) & C_3 &= \{S, I, G\} \\
 \tau_4(G, J) &= \sum_H \phi_H(H, G, J) & C_4 &= \{H, G, I\} \\
 \tau_5(S, L, J) &= \sum_G \phi_L(L, G) \tau_3(S, G) \tau_4(G, J) & C_5 &= \{L, G, S, J\} \\
 \tau_6(L, J) &= \sum_S \phi_J(J, L, S) \tau_5(S, L, J) & C_6 &= \{J, L, S\}
 \end{aligned}$$



-Tree

-Clusters are Clique

Example



$$\tau_1(D) = \sum_C \phi_C(C) \phi_D(C, D)$$

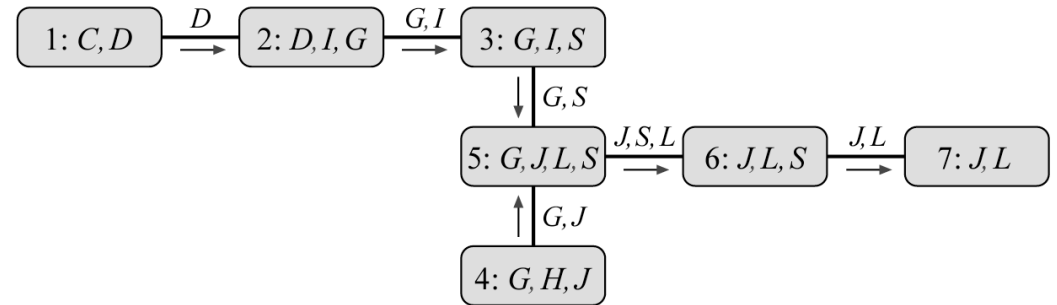
$$\tau_2(G, I) = \sum_D \phi_G(G, I, D) \tau_1(D)$$

$$\tau_3(S, G) = \sum_I \phi_S(S, I) \phi_I(I) \tau_2(G, I)$$

$$\tau_4(G, J) = \sum_H \phi_H(H, G, J)$$

$$\tau_5(L, J) = \sum_G \phi_L(L, G) \tau_3(S, G) \tau_4(G, J)$$

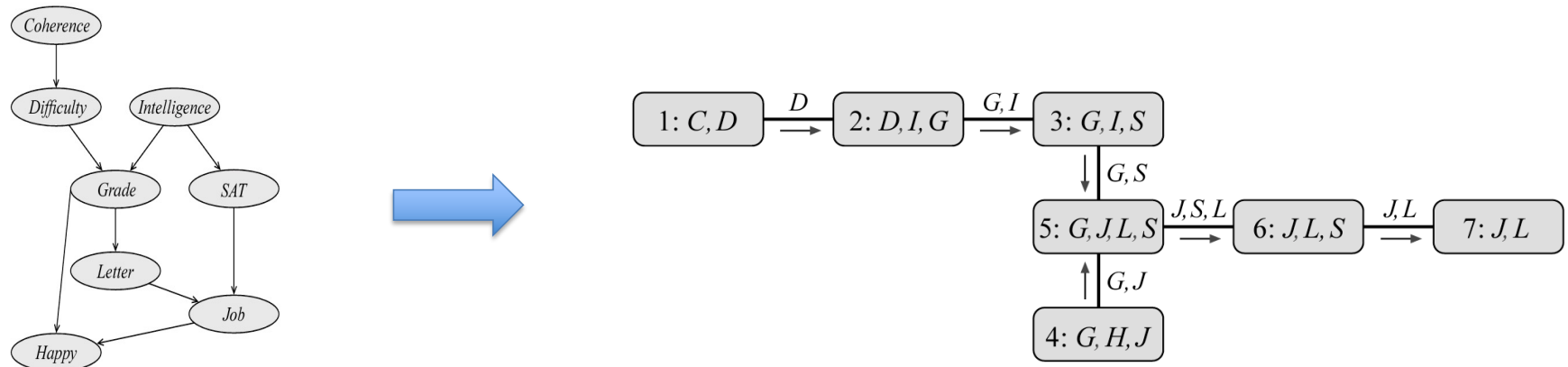
$$\tau_6 = \sum_{L, S} \phi_J(J, L, S) \tau_5(L, J)$$



- Execution of VE defines a direction for the edges
- Adjacent clusters pass information to each other about variables in sepsets
- All the messages flowing forward a single cluster where the final result is (root)

Cluster tree

- Partitions a graph into clusters of variables
 - the variables within a cluster could be highly coupled
 - interactions among clusters have a tree structure
 - cluster will be only directly influenced by its neighbors in the tree.

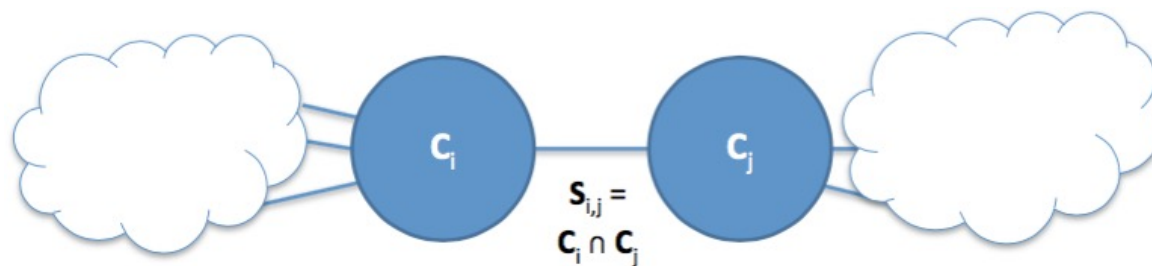


Cluster tree

- Variable elimination induces a cluster tree
 - Each τ_i is used once
 - used in sum-product to generate ψ_i , then it is removed
 - Execution of VE defines a direction for the edges
- A cluster tree induced by VE satisfies the running intersection properties (No loop)
- A cluster tree that satisfies running intersection property is a clique tree (“Junction Tree,” or “Join Tree”)

Clique tree

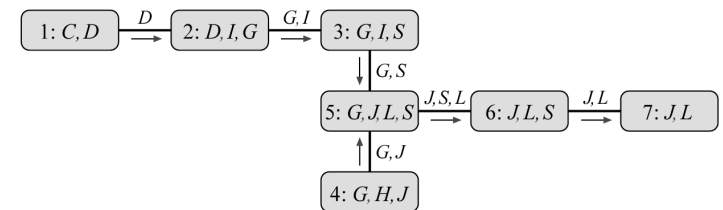
- Given an undirected graph H , a tree T is a clique tree for H if:
 - Each node in T corresponds to a clique in H
 - Each maximal clique in H is a node in T
 - Each sepset $\mathbf{S}_{i,j}$ separates the variables strictly on one side of the edge from the variables on the other side.



Clique tree and VE

- VE induces a clique tree
- Clique tree provides a data structure for efficient execution of VE
 - Each product-factor ψ_i belongs to a clique-vertex
 - Each τ_i is a “message” from one clique-vertex to another.

$$\begin{aligned}\tau_1(D) &= \sum_C \phi_C(C) \phi_D(C, D) \\ \tau_2(G, I) &= \sum_D \phi_G(G, I, D) \tau_1(D) \\ \tau_3(S, G) &= \sum_I \phi_S(S, I) \phi_I(I) \tau_2(G, I) \\ \tau_4(G, J) &= \sum_H \phi_H(H, G, J) \\ \tau_5(L, J) &= \sum_G \phi_L(L, G) \tau_3(S, G) \tau_4(G, J) \\ \tau_6 &= \sum_{L, S} \phi_J(J, L, S) \tau_5(L, J)\end{aligned}$$

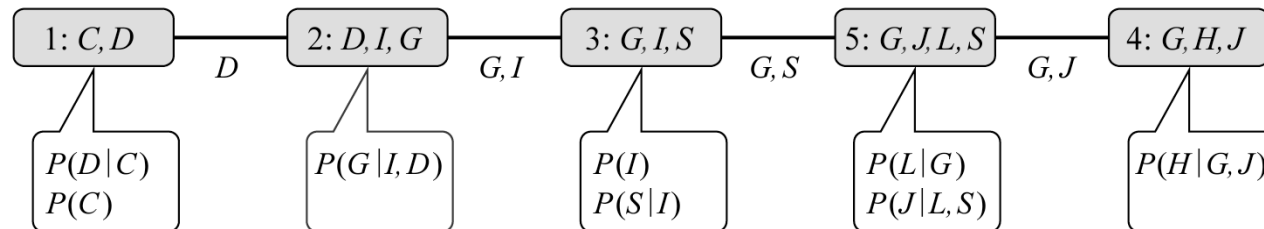
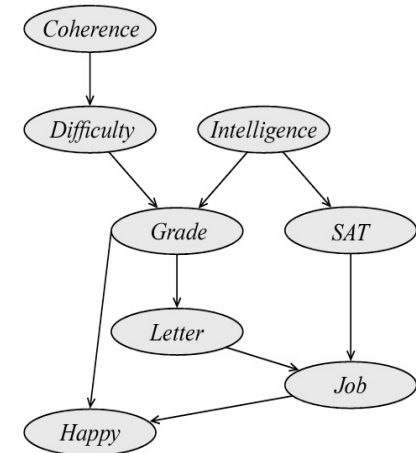


A clique tree is given, how this data structure can be used to perform VE?

Clique tree and VE

$$P(C, D, I, G, S, L, J, H)$$

$$= P(C)P(D | C)P(I)P(G | I, D)P(S | I) \\ P(L | G)P(J | L, S)P(H | G, J)$$



How can we do VE?

Considering flow of information or message passing

Clique tree message passing

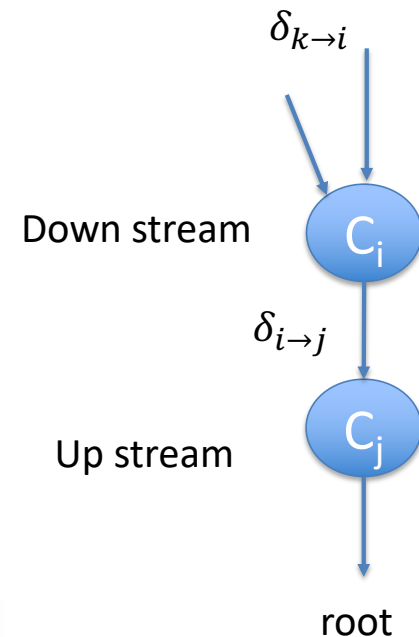
VE: message passing

- Generate a set of initial potentials for each clique node $\psi_j(C_j)$

$$\psi_j(\mathbf{C}_j) = \prod_{\phi : \alpha(\phi)=j} \phi$$

- Choose an arbitrary root clique $\mathbf{C}_r$
 - This imposes a directionality on the graph
- Pass messages “upstream”

$$\delta_{i \rightarrow j} = \sum_{\mathbf{C}_i - \mathbf{S}_{i,j}} \psi_i \cdot \prod_{k \in (\text{Nb}_i - \{j\})} \delta_{k \rightarrow i}$$

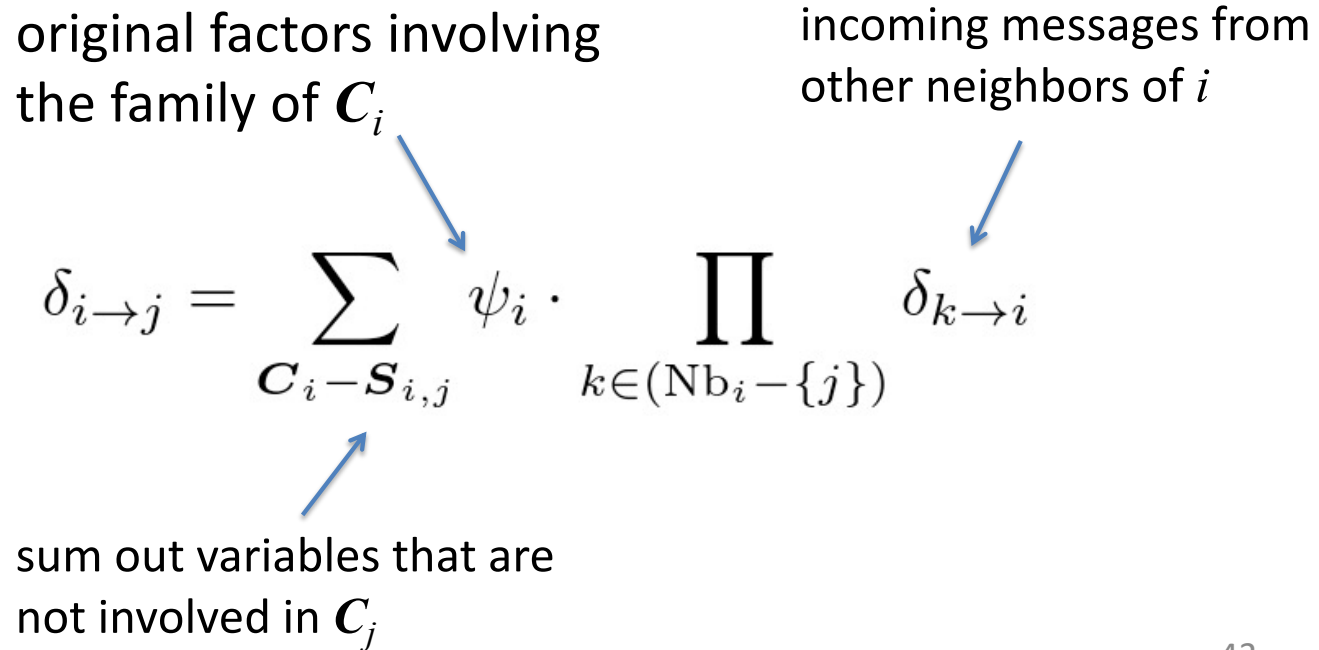


Message passing in the clique tree

All about passing messages and obtaining the final information $\rightarrow$ belief

original factors involving
the family of $\mathbf{C}_i$

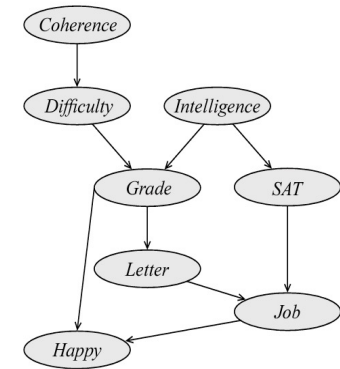
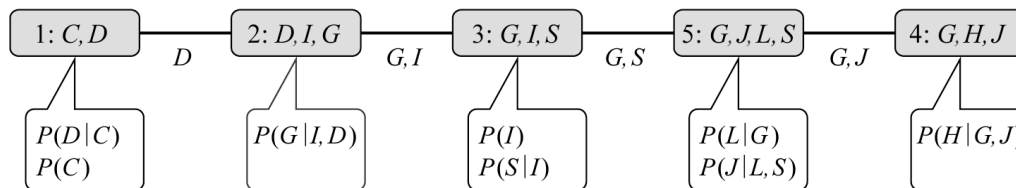
incoming messages from
other neighbors of i

$$\delta_{i \rightarrow j} = \sum_{\mathbf{C}_i - \mathbf{S}_{i,j}} \psi_i \cdot \prod_{k \in (\text{Nb}_i - \{j\})} \delta_{k \rightarrow i}$$
The diagram shows the equation $\delta_{i \rightarrow j} = \sum_{\mathbf{C}_i - \mathbf{S}_{i,j}} \psi_i \cdot \prod_{k \in (\text{Nb}_i - \{j\})} \delta_{k \rightarrow i}$. Three blue arrows point from explanatory text to parts of the equation: one from 'original factors involving the family of $\mathbf{C}_i$ ' to ψ_i , one from 'incoming messages from other neighbors of i ' to $\delta_{k \rightarrow i}$, and one from 'sum out variables that are not involved in $\mathbf{C}_j$ ' to the summation term $\sum_{\mathbf{C}_i - \mathbf{S}_{i,j}}$.

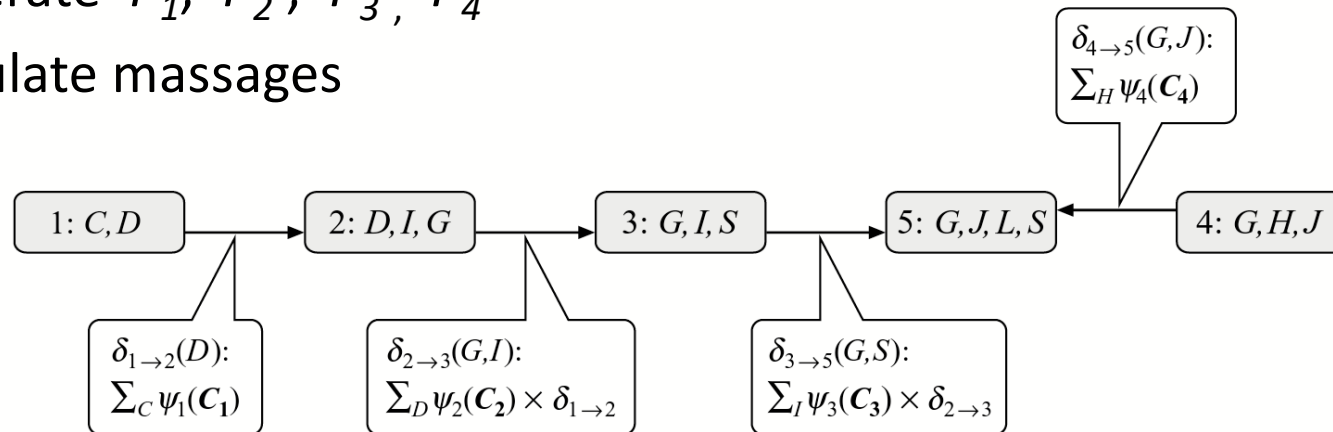
sum out variables that are
not involved in $\mathbf{C}_j$

Example

- $P(J)$, $C(5)$ is the root

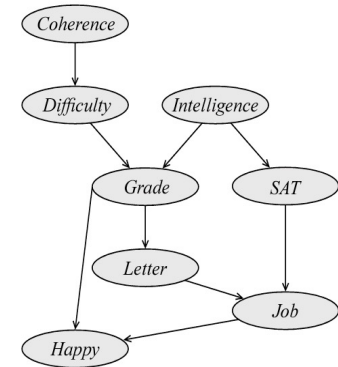
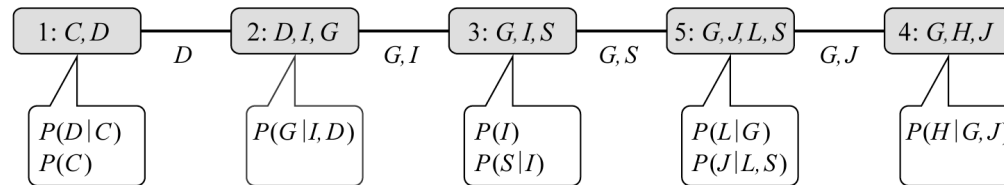


- Generate $\psi_1, \psi_2, \psi_3, \psi_4$
- Calculate messages

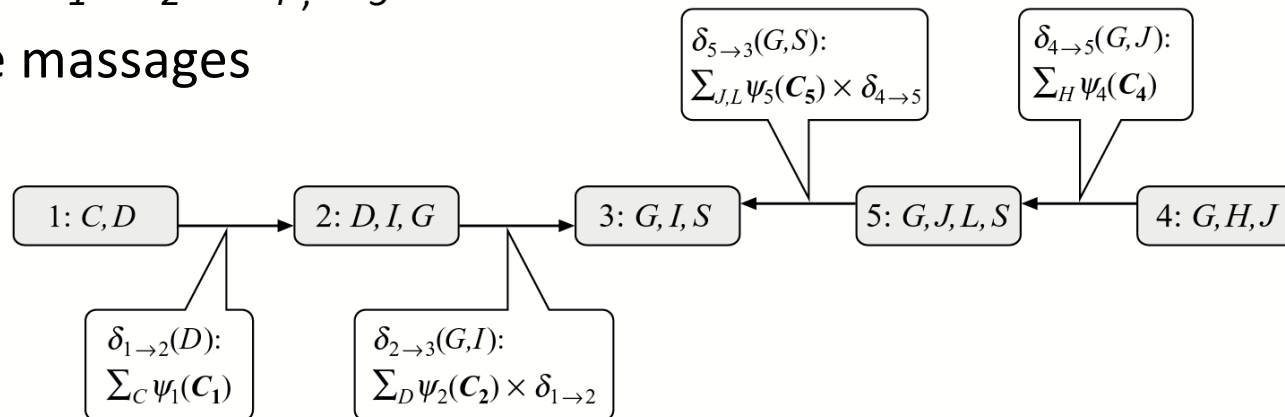


Example

- $P(G)$, $C(3)$ is the root



- Generate $\psi_1, \psi_2, \psi_4, \psi_5$
- Calculate messages



Message passing in the clique tree

Similar to the VE algorithm

ψ : multiply of factors

τ : new factor by normalization

$$\tau_1(D) = \sum_C \phi_C(C) \phi_D(C, D)$$

$$\tau_2(G, I) = \sum_D \phi_G(G, I, D) \tau_1(D)$$

$$\tau_3(S, G) = \sum_I \phi_S(S, I) \phi_I(I) \tau_2(G, I)$$

$$\tau_4(G, J) = \sum_H \phi_H(H, G, J)$$

$$\tau_5(L, J) = \sum_G \phi_L(L, G) \tau_3(S, G) \tau_4(G, J)$$

$$\tau_6 = \sum_{L, S} \phi_J(J, L, S) \tau_5(L, J)$$

$$\delta_{i \rightarrow j} = \sum_{C_{i-S_{i,j}}} \psi_i \cdot \prod_{k \in (\text{Nb}_i - \{j\})} \delta_{k \rightarrow i}$$

Message Passing in the Clique Tree

- Recursive structure: message-sum of products of message-sums of products of message-sums of ...
- work “upward”
- Root will be the last

$$\delta_{i \rightarrow j} = \sum_{C_i - S_{i,j}} \psi_i \cdot \prod_{k \in (\text{Nb}_i - \{j\})} \delta_{k \rightarrow i}$$

How can we obtain inference for a variable?

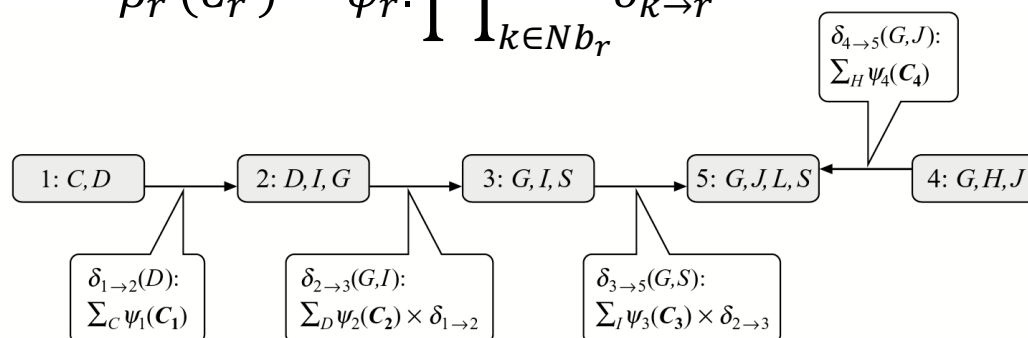
Beliefs

- When the root clique receives all messages, the final factor (belief) can be calculated

$$\beta_r(\mathbf{C}_r) = \tilde{P}_{\Phi}(\mathbf{C}_r) = \sum_{\mathcal{X}-\mathbf{C}_r} \prod_{\phi} \phi$$

$$\beta_r(\mathbf{C}_r) = \psi_r \cdot \prod_{k \in Nb_r} \delta_{k \rightarrow r}$$

Example:



$$\beta_5(G, L, J, S) = \delta_{3 \rightarrow 5} \delta_{4 \rightarrow 5} \psi_5(G, L, J, S)$$

How can we obtain $P(J)$?

The Algorithm

- Input: clique tree T , factors Φ , root C_r
- For each clique C_i , calculate ψ_i
- While C_r is still waiting on incoming messages:
 - Choose a C_i that has received all of its incoming messages.
 - Calculate and send the message from C_i to $C_{\text{upstream-neighbor}(i)}$
- Return β_i

$$\beta_i \leftarrow \psi_i \cdot \prod_{k \in \text{Nb}_i} \delta_{k \rightarrow i}$$

Obtaining marginals

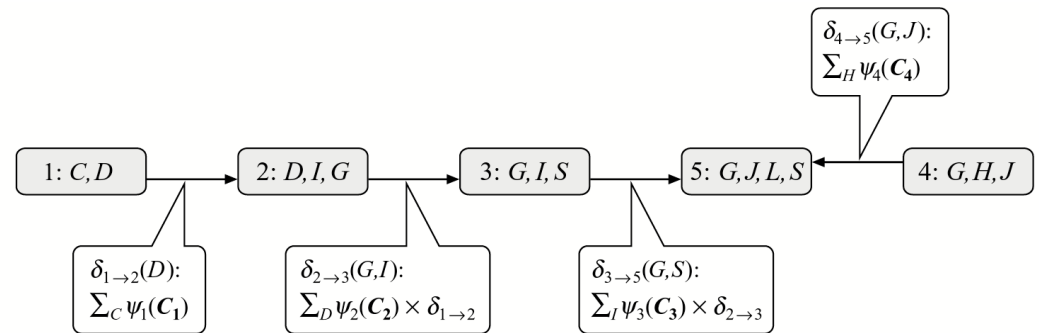
Given a set of query variables $\mathbf{Q}$ that is contained in some clique $\mathbf{C}$:

- Make that clique the root.
- Run the upward pass of clique tree VE, resulting in factor β .
- Return $\sum_{\mathbf{C}-\mathbf{Q}} \beta$.

Example:

$$\beta_5(G, L, J, S) = \delta_{3 \rightarrow 5} \delta_{4 \rightarrow 5} \psi_5(G, L, J, S)$$

$$\tilde{P}(J) = \sum_{G, L, S} \delta_{3 \rightarrow 5} \delta_{4 \rightarrow 5} \psi_5(G, L, J, S)$$



How can we obtain inference for all variables?

Computing all posterior probabilities

We can modify this algorithm to give us the marginal probability of every random variable in the network.

- Naively: run clique tree VE with each clique as the root.
- Better: notice that the same messages get used on different runs; only two messages per clique tree edge.

